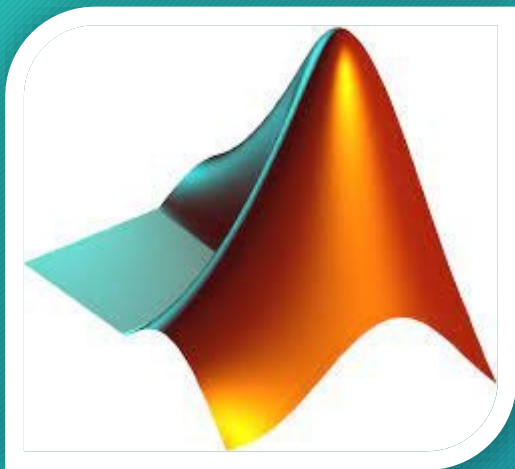




Communication Design

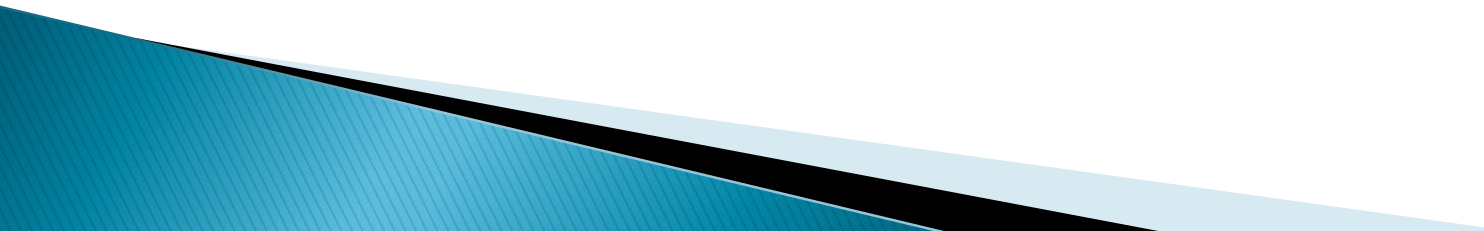
LAB.3

Preprocessing Signals

Ass.Lec.Ammar Alabbassi

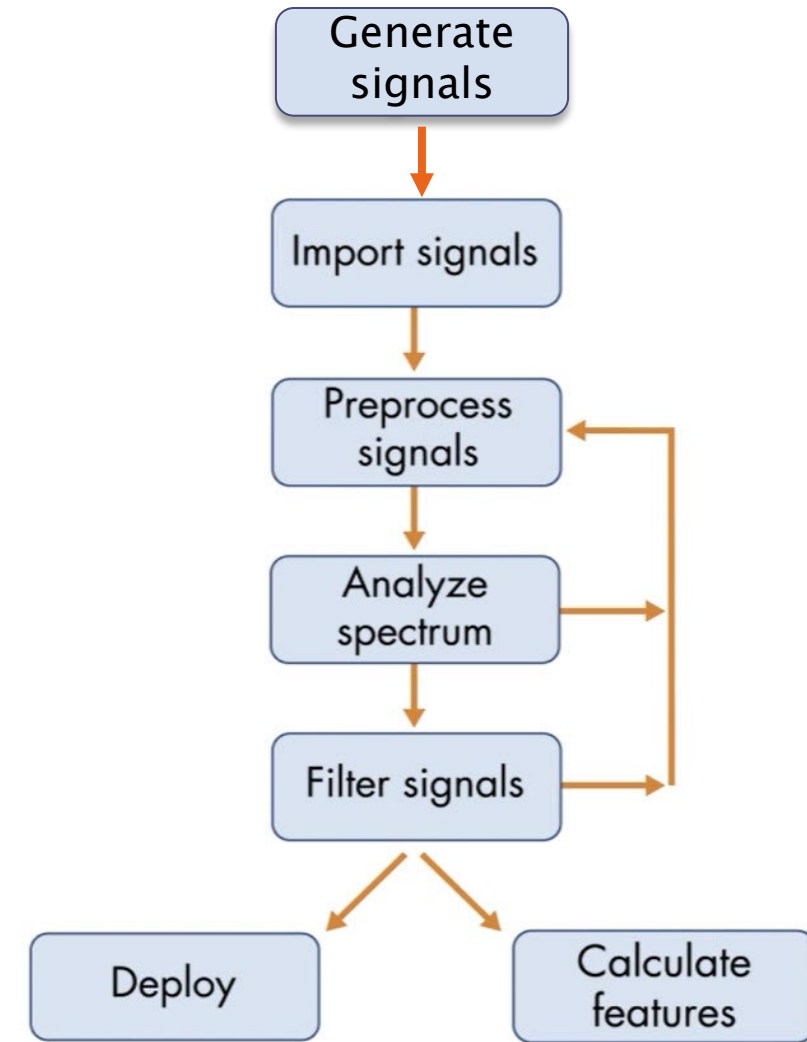
Outline:

Preprocessing signals

- ▶ Resample Signals
 - ▶ Extract Interesting Regions
 - ▶ Custom Preprocessing Function
 - ▶ Align Signals
- 

Signal Processing Overview

1. Generate signals.....done
2. Import the signals into MATLAB.....done
3. Preprocess the signals in the time domain to prepare them for analysis
4. Look at the power spectrum in the frequency domain
5. Filter the signals to keep only relevant frequency content
6. Revisit earlier steps in the process to refine your algorithm
7. Extract information from the signals or deploy your algorithm to hardware



Quiz

- 1– Specify a sample rate of **300 Hz** in a variable named **fs**. Then create a signal named **(t)** that extends from **0 seconds to 15 second**, with time steps that are **1/fs**.
- 2– Create a **2 Hz** sine signal named **(sig)** at the sample times in **t**.
- 3– Create a vector of normally distributed random numbers named **noise** that is the same size as **(sig)** and will be added to the original signal **(sig)** to generate a new signal named **(signoise)** that makes distortion about 20% of the original signal **(Sig)**.
- 4– plot the original signal **(sig)** and noisy signal **(signoise)**.
- 5– Finally, import them into the signal analyzer and view the signals in time and frequency domains .

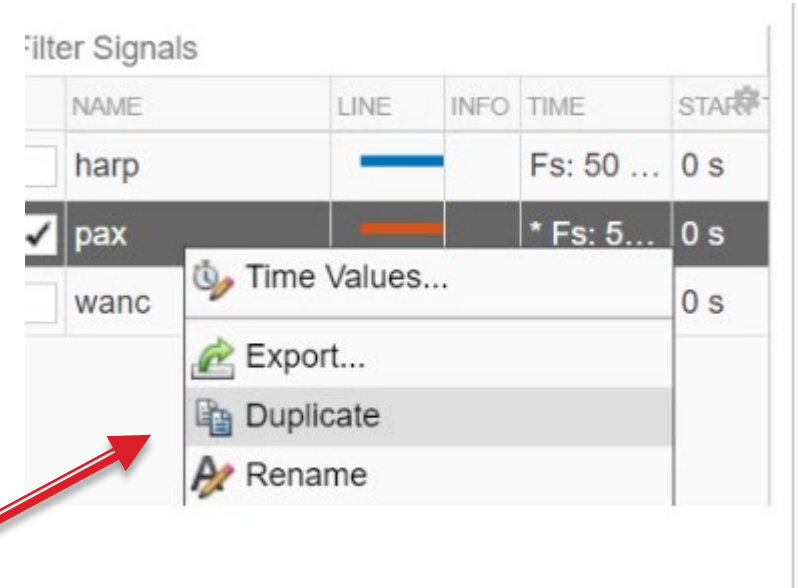
Resample Signals



Task 1

Before you modify a signal, it is best practice to **duplicate** the signal so that you can compare the signal before and after processing. To process a signal, follow these steps:

1 – Duplicate the **pax** signal by right-clicking it in the Signal table and selecting Duplicate.



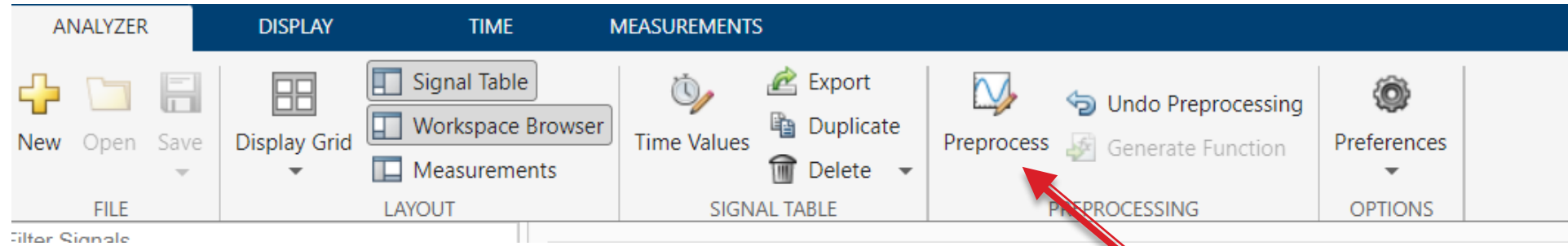
2 – Rename the duplicate **pax_uniform** by right-clicking it and selecting Rename.

The screenshot shows the 'Filter Signals' table after the duplication step. The 'pax_uniform' signal is now present at the bottom of the table and is selected (checked). A red arrow points from the text 'Rename the duplicate pax_uniform' to the 'pax_uniform' row.

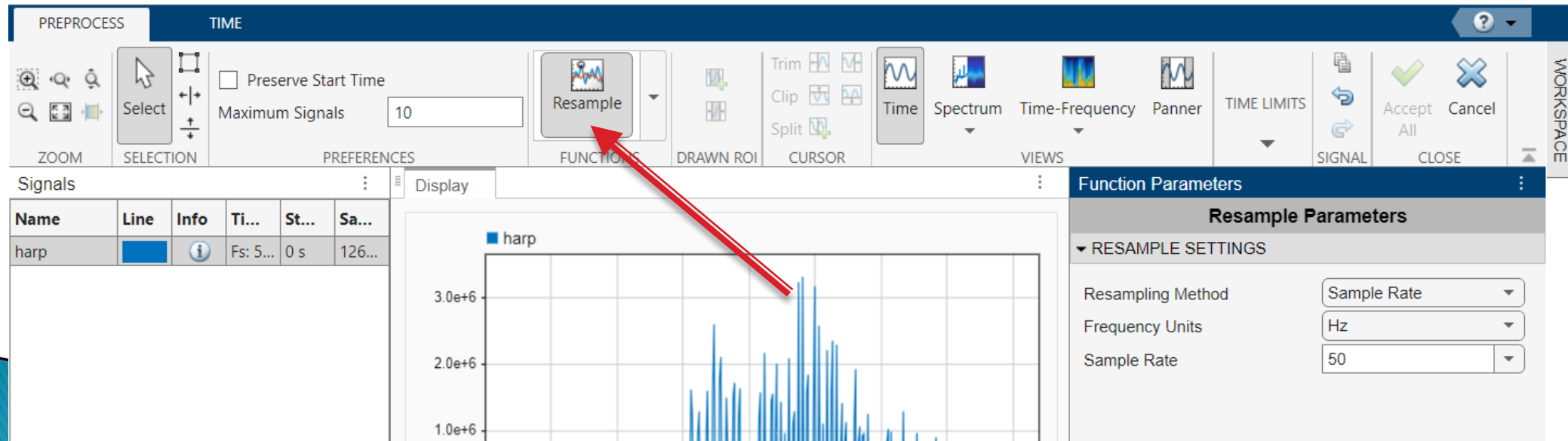
	NAME	LINE	INFO	TIME	START
☐	harp			Fs: 50 ...	0 s
☒	pax			* Fs: 5...	0 s
☐	wanc			Fs: 10...	0 s
☒	pax_uniform			* Fs: 5...	0 s

Task 1

3– Click Preprocess in the Analyzer tab of toolstrip to enter preprocessing mode

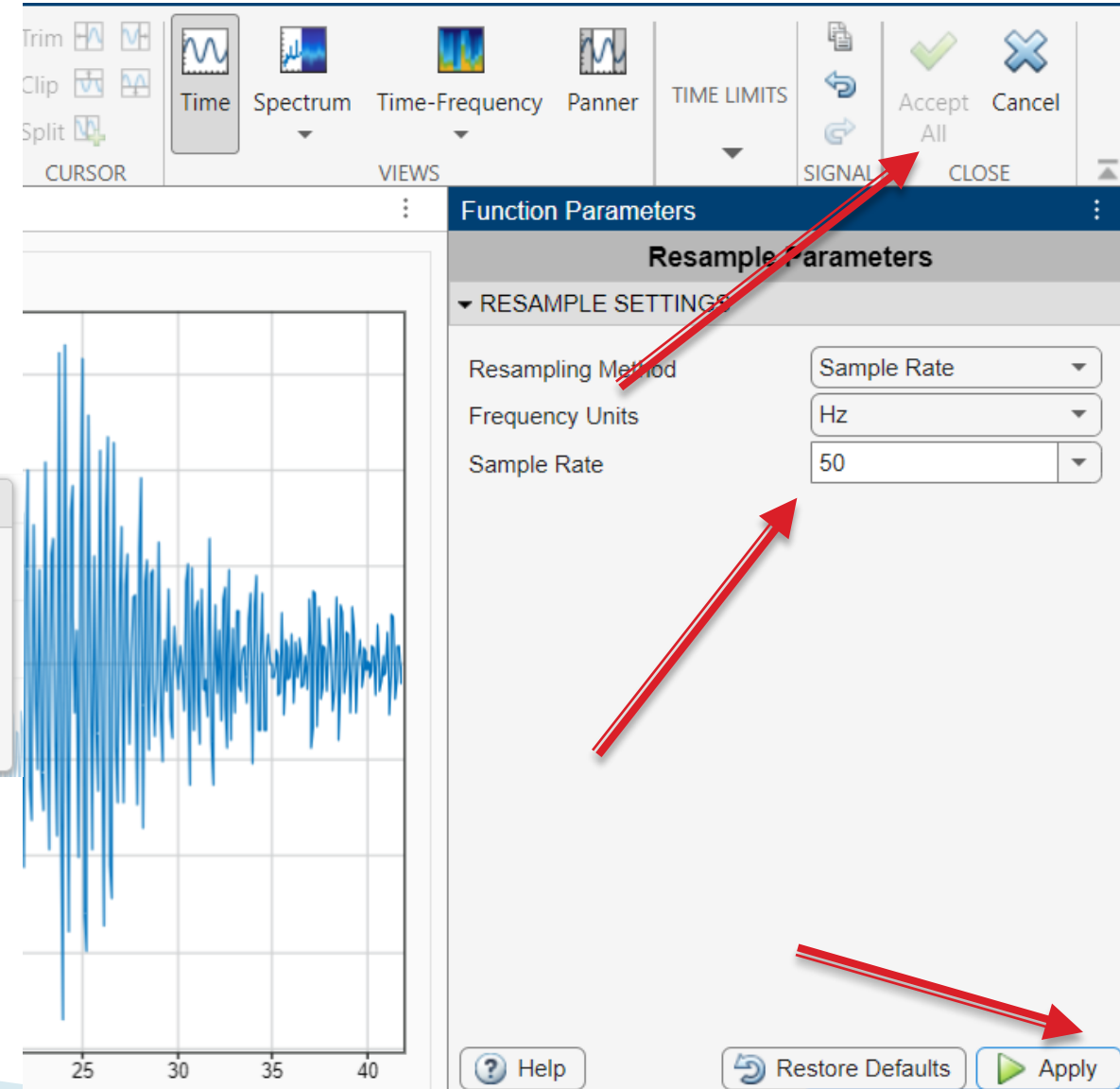
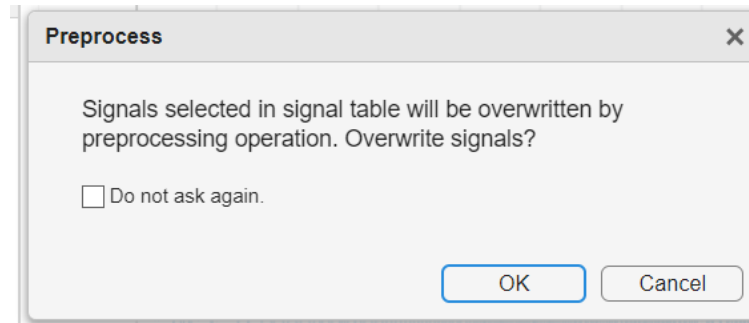


4– In the Functions section, select Resample



Task 1

- 5– Set the sample rate to 50 Hz in the Function Parameters panel
- 6– Click Apply
- 7– Click Ok on the Preprocess warning dialog
- 8– Click Accept All in the Close section of the toolbar to leave preprocessing mode

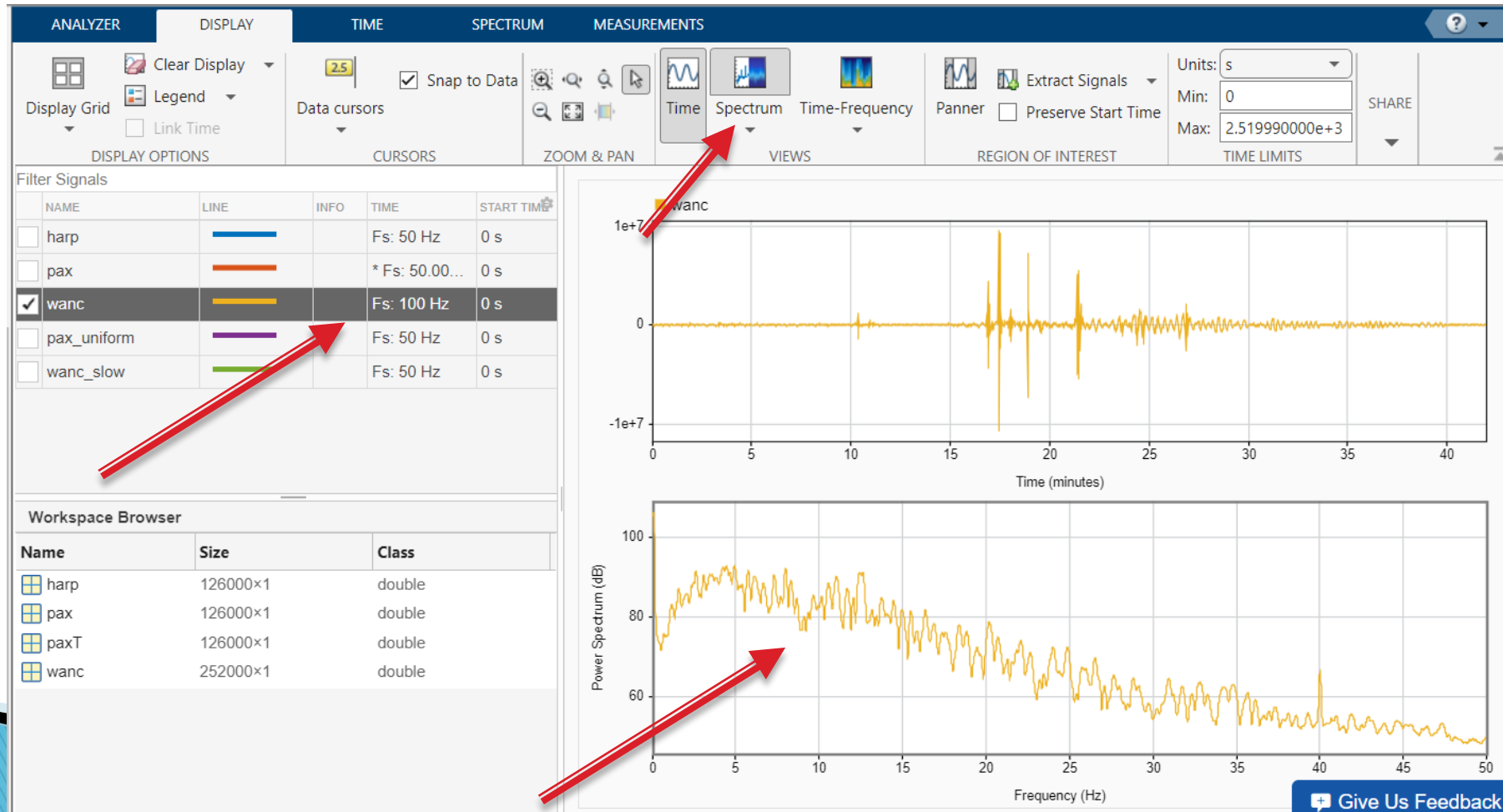


Task 2

The signal recorded at the **Wrangell** station has a **faster sample rate of 100 Hz**.

TASK

Look at the full frequency range of the **wanc** signal's power spectrum.



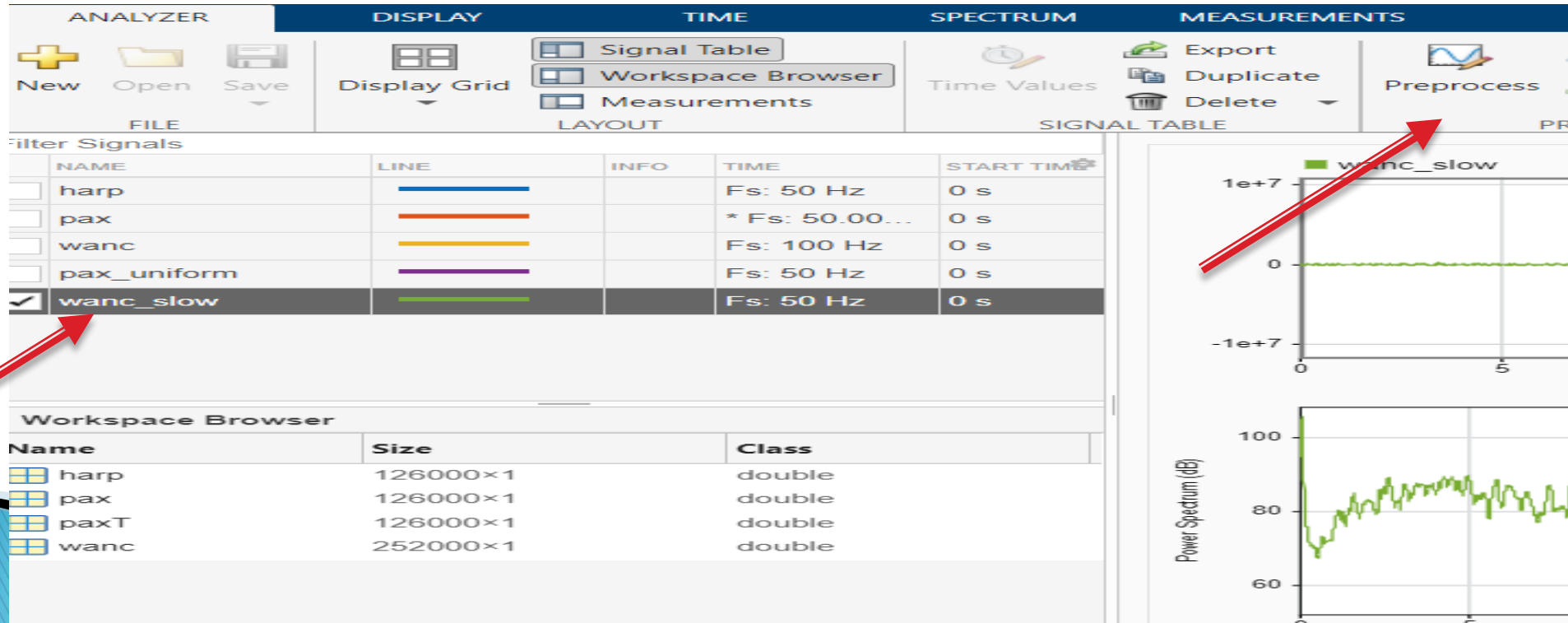
Task 3

1-When you resample to a **slower sample rate**, frequency content **above $fs/2$** , where **fs** is the sample rate after resampling, can **distort the power spectrum**.

2- In the **WANC** signal, there isn't significant power **above 25 Hz**, so you can try resampling the WANC signal **to 50 Hz** so that it matches the sample rate of the HARP and PAX signals.

TASK

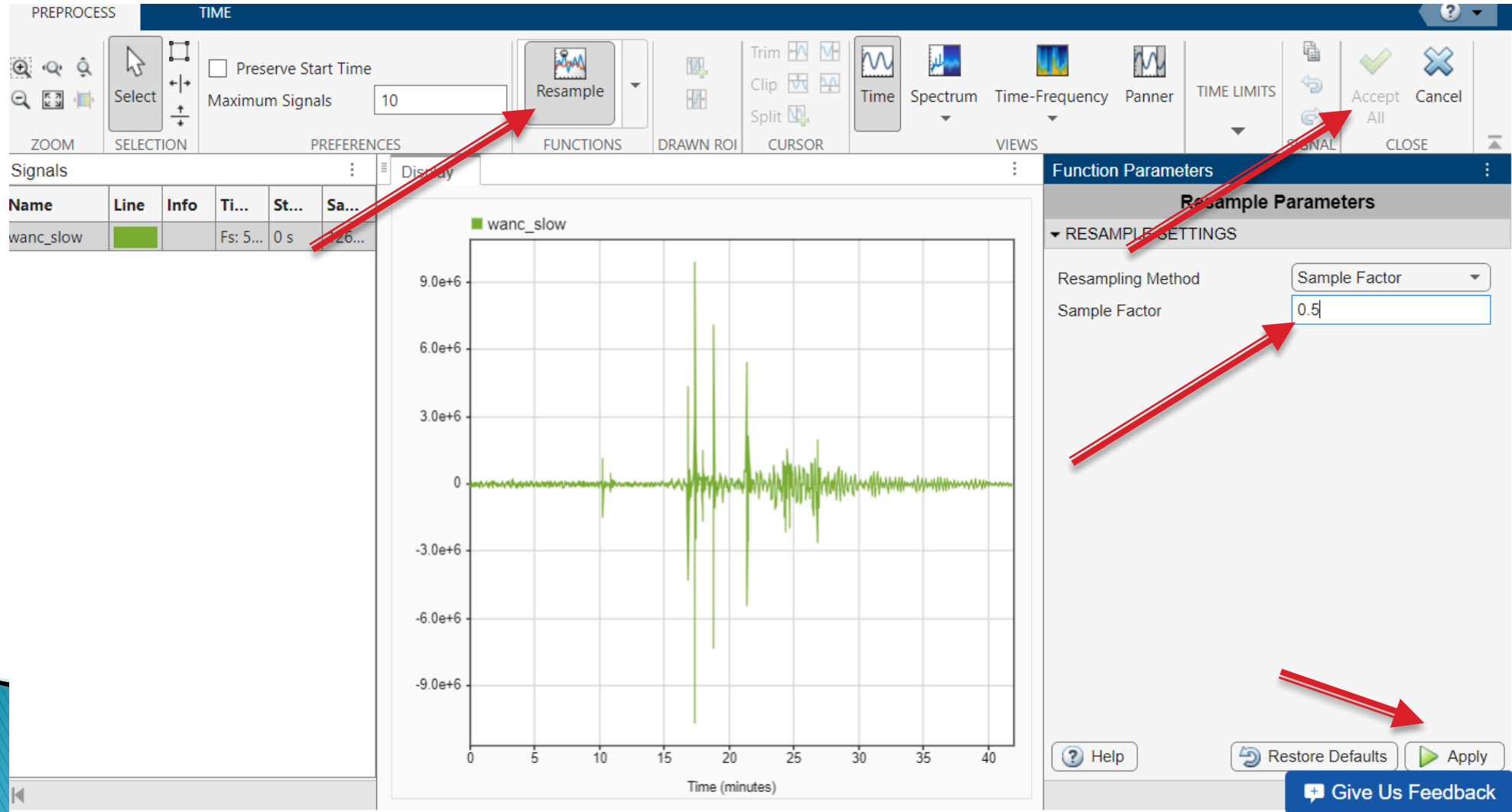
- * Duplicate **wanc** and rename the copy **wanc_slow**. Then **resample wanc_slow** by a **Sample Factor of 0.5**.
- * Display the original and resampled signals to compare the effects of resampling.



Task 3

TASK

- * Duplicate **wanc** and rename the copy **wanc_slow**. Then **resample wanc_slow** by a **Sample Factor of 0.5**.
- * Display the original and resampled signals to compare the effects of resampling.

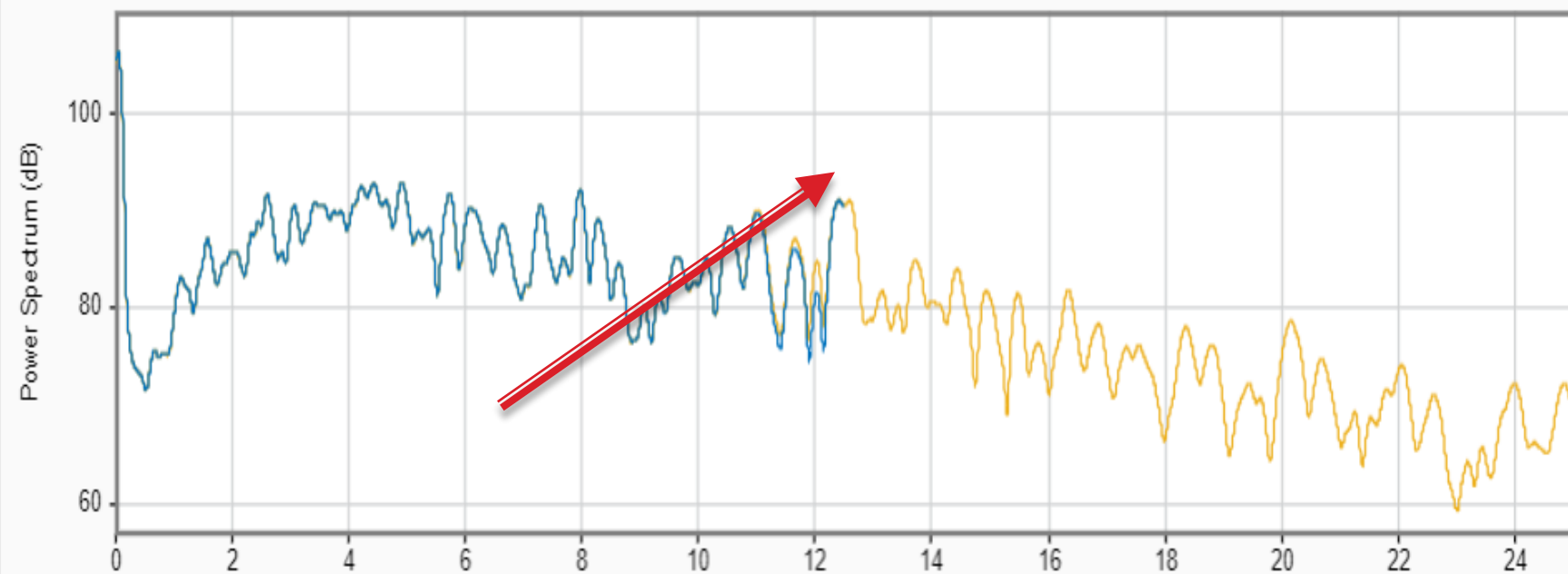
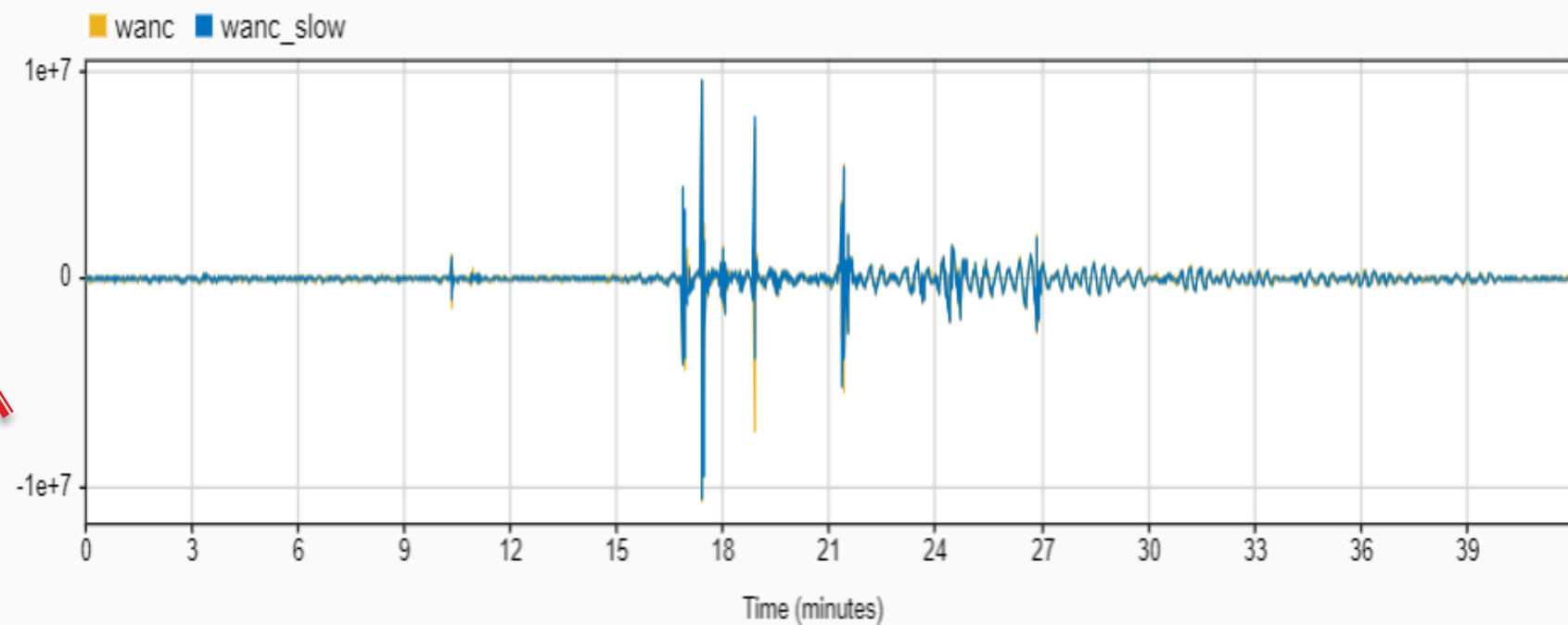


Filter Signals

	NAME	LINE	INFO	TIME	START
☐	harp			Fs: 50 Hz	0 s
☐	pax			* Fs: 50....	0 s
☒	wanc			Fs: 100 Hz	0 s
☐	pax_uniform			Fs: 50 Hz	0 s
☒	wanc_slow			Fs: 50 Hz	0 s

Workspace Browser

Name	Size	Class
 harp	126000×1	double
 pax	126000×1	double
 paxT	126000×1	double
 wanc	252000×1	double



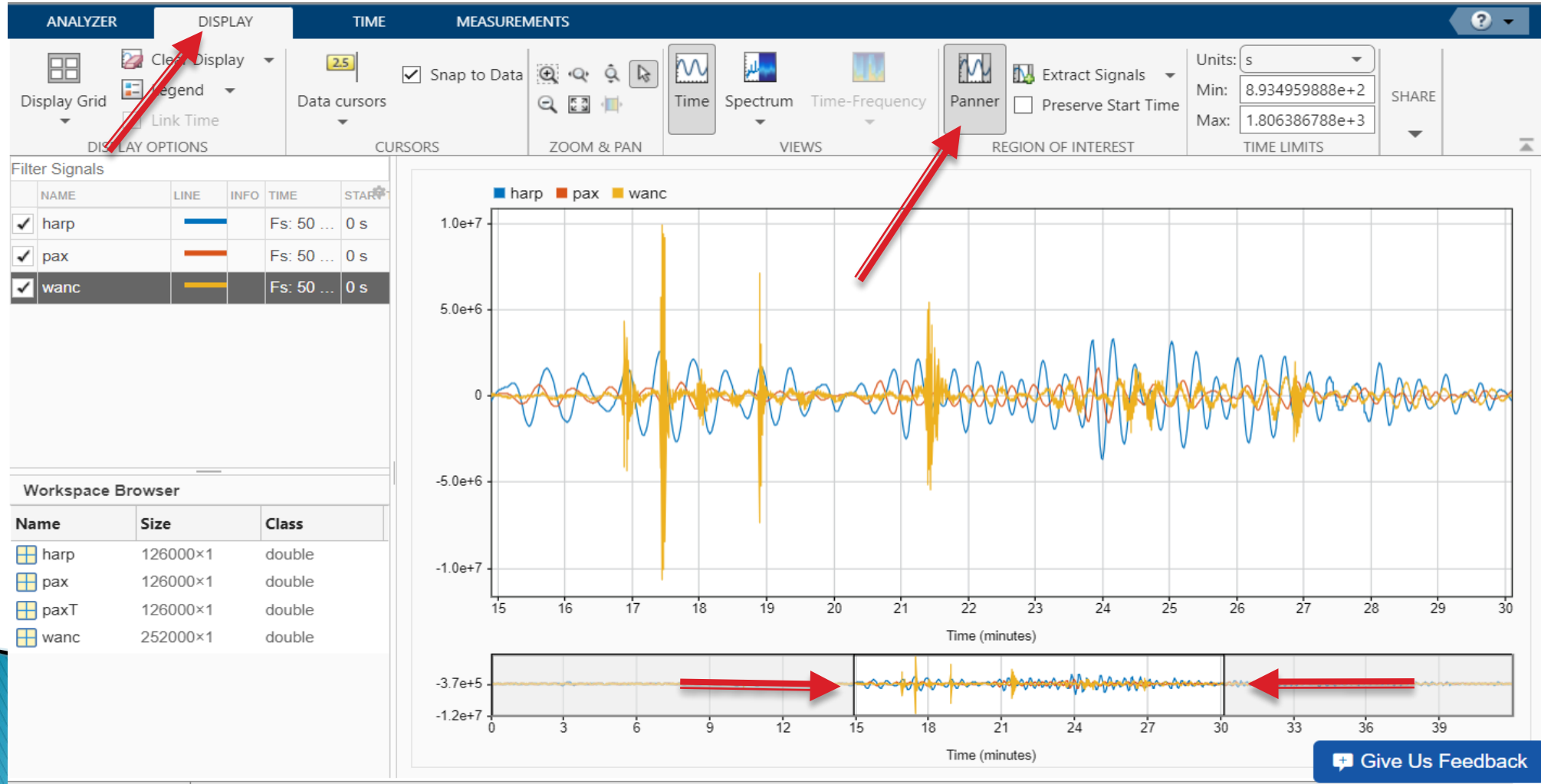
Extract Interesting Regions

Task 1

Most of the activity in the signals from the **HARP** and **PAX** stations occurs in a smaller portion of the signal. You can zoom in on interesting regions. On the Display tab, locate the Region of Interest section and select the Panner tool.

#TASK

Use the **panner** to display about **12 minutes to 18 minutes**.

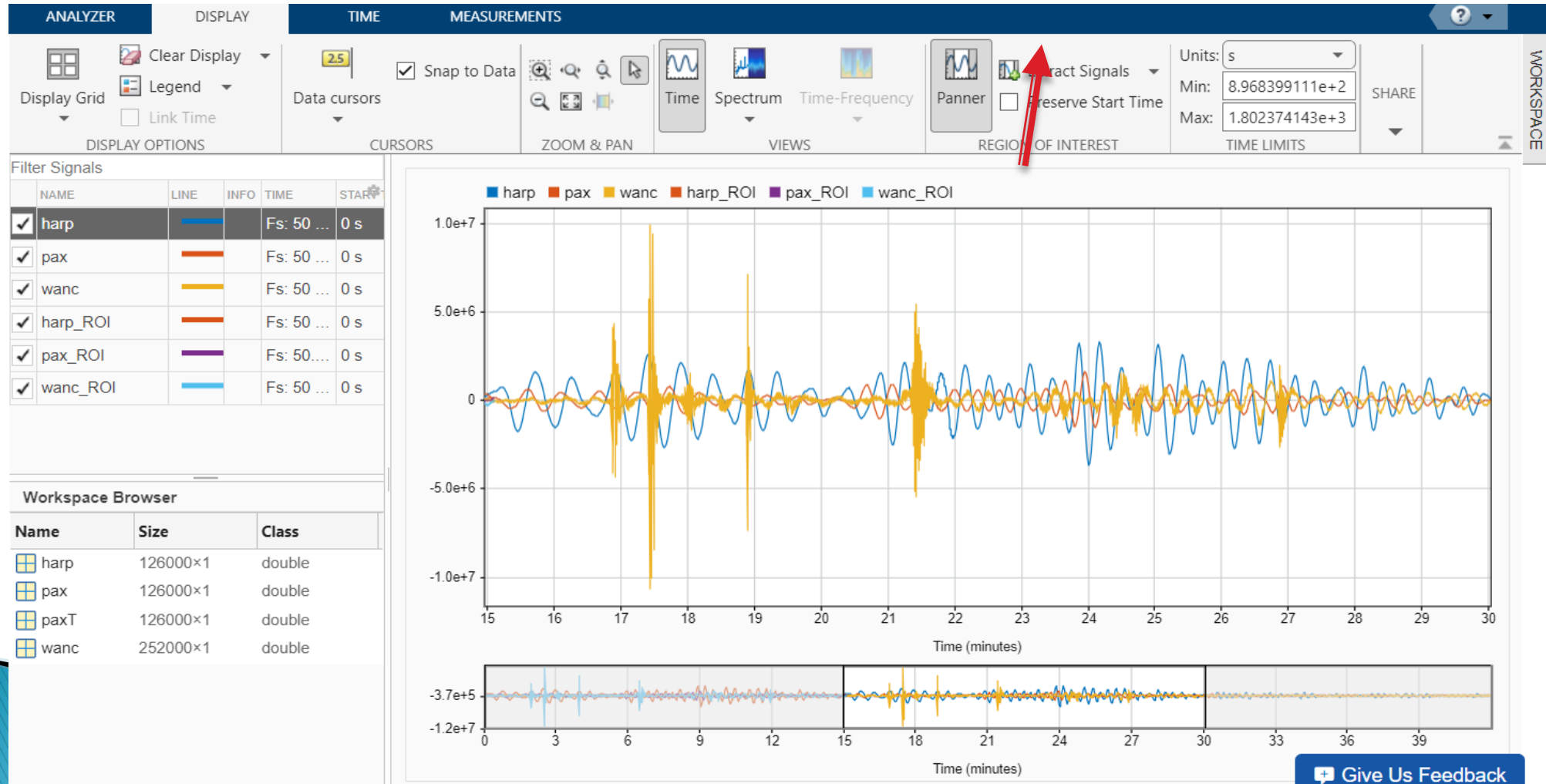


Task 2

You can extract the data from the surface waves by removing extra points before and after. On the **Display tab**, you can find the **Extract Signals** option in the **Region of Interest** section.

TASK

Extract the selected region from the **harp**, **pax**, and **wanc** signals. Make sure that the extracted signals are named **harp_ROI**, **pax_ROI**, and **wanc_ROI** respectively and display them.



Custom Preprocessing Function

Task 1

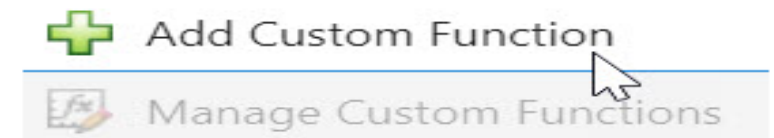
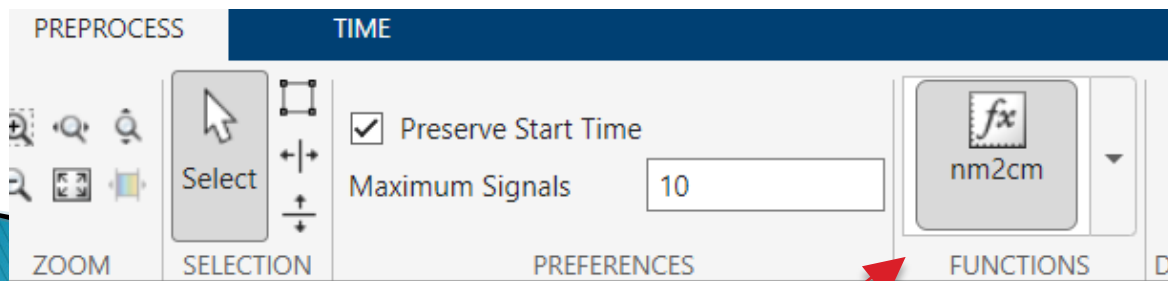
The **y-axis** of the seismic signals is displacement in **nanometers**, but it can be easier to interpret the displacement in **centimeters**. If you want to preprocess your signals with a method not built into Signal Analyzer, you can add a **custom function**. Once you're in **preprocessing mode**, select **Add Custom Function** at the bottom of the **Functions dropdown**. A function named **nm2cm** has been provided to convert the signals **from nanometers to centimeters**. so the function file already contains.

```
function [y,t] = nm2cm(x,t)
    y = x/1e7;
End
```

TASK

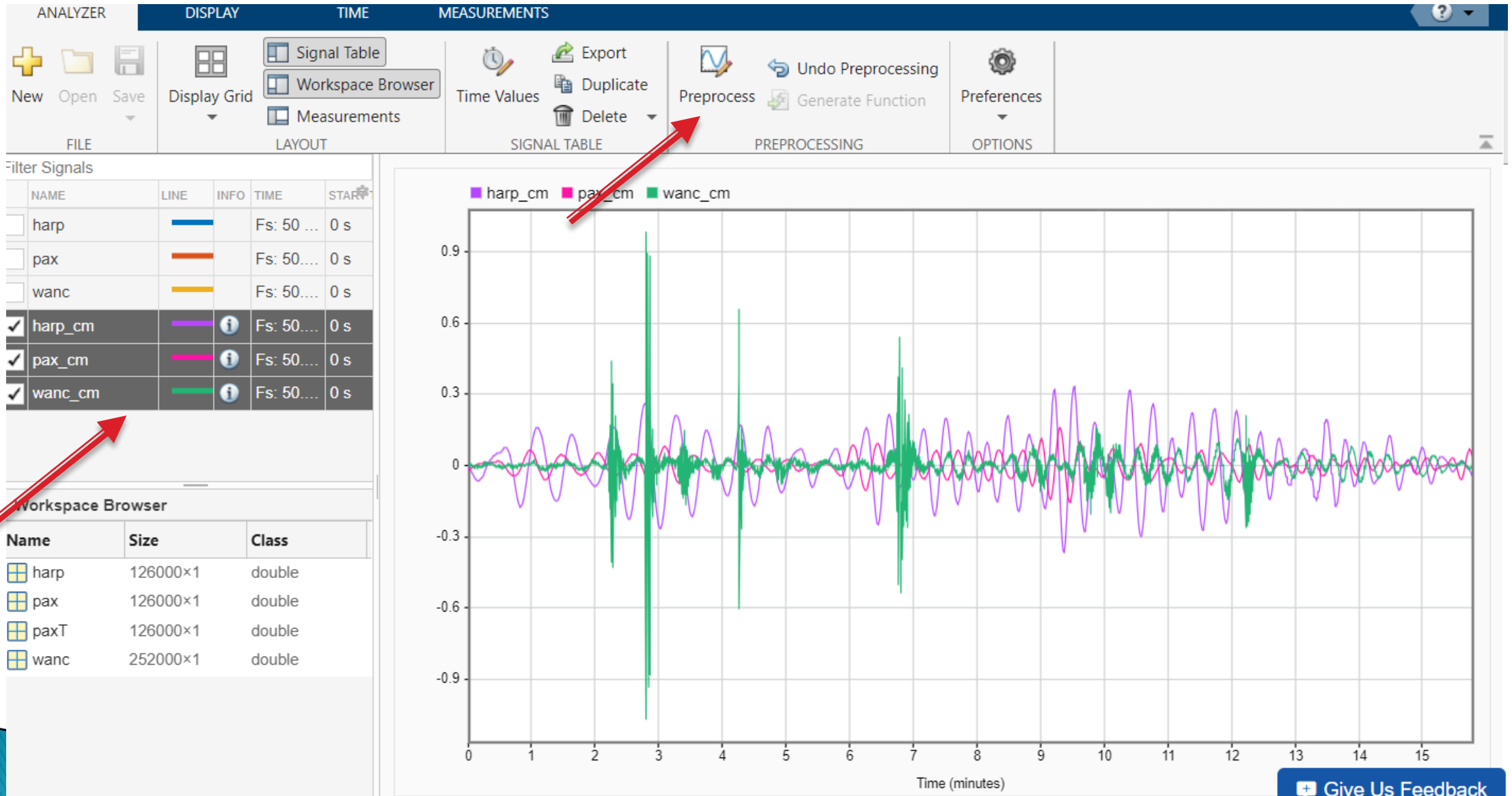
Duplicate the **harp, pax, and wanc** signals and **append _cm** to the signal names. For example, duplicate harp and name the result harp_cm.

The function **nm2cm** has been written for you. Use the custom preprocessing function nm2cm to process the three new signals so that their units are in centimeters instead of nanometers. Then display the three new signals in the time domain.



*In preprocessing mode, click **Add Custom Function**.*

Task 1



Task 1

The screenshot displays the Signal Analyzer software interface. A dialog box titled "Add Custom Function" is open, allowing the user to add a new function to the preprocessing gallery. The dialog box contains the following fields and text:

- Name:** nm2cm
- Description:** My custom function with optional input arguments 'arg1,arg2...'

A red arrow points to the "Name" field. Below the input fields, the dialog box provides instructions on how to add a function:

Add a function with the following syntax to the Signal Analyzer preprocessing gallery.

$$[y, tOut] = \text{Name}(x, tIn, varargin)$$

x, the selected signal, and tIn, its time vector, are automatically passed into the function. An optional input argument list can also be specified.

The dialog box has "OK" and "Cancel" buttons at the bottom right.

The background interface shows a table of signals and a plot of a signal over time.

Name	Line	Info	Time	St...
harp_cm			Fs: 50.0000 Hz	0 s
pax_cm			Fs: 50.0000 Hz	0 s
wanc_cm			Fs: 50.0000 Hz	0 s

The plot shows a signal over time (minutes) with a y-axis ranging from -0.9 to -0.6. The x-axis ranges from 0 to 14 minutes.

Task 1

PREPROCESS TIME

SELECT ☐ Preserve Start Time
Maximum Signals 10

FUNCTIONS **nm2cm** DRAWN ROI CURSOR VIEWS

TIME LIMITS SIGNAL **Accept All** **Cancel** **CLOSE**

WORKSPACE

Signals

Name	Line	Info	Time	St...	Sample
harp_cm			Fs: 50.0000 Hz	0 s	47358
pax_cm			Fs: 50.0000 Hz	0 s	47358
wanc_cm			Fs: 50.0000 Hz	0 s	47358

Display

harp_cm pax_cm wanc_cm

0.9
0.6
0.3
0
-0.3
-0.6
-0.9

0 2 4 6 8 10 12 14

Time (minutes)

Function Parameters

nm2cm Parameters

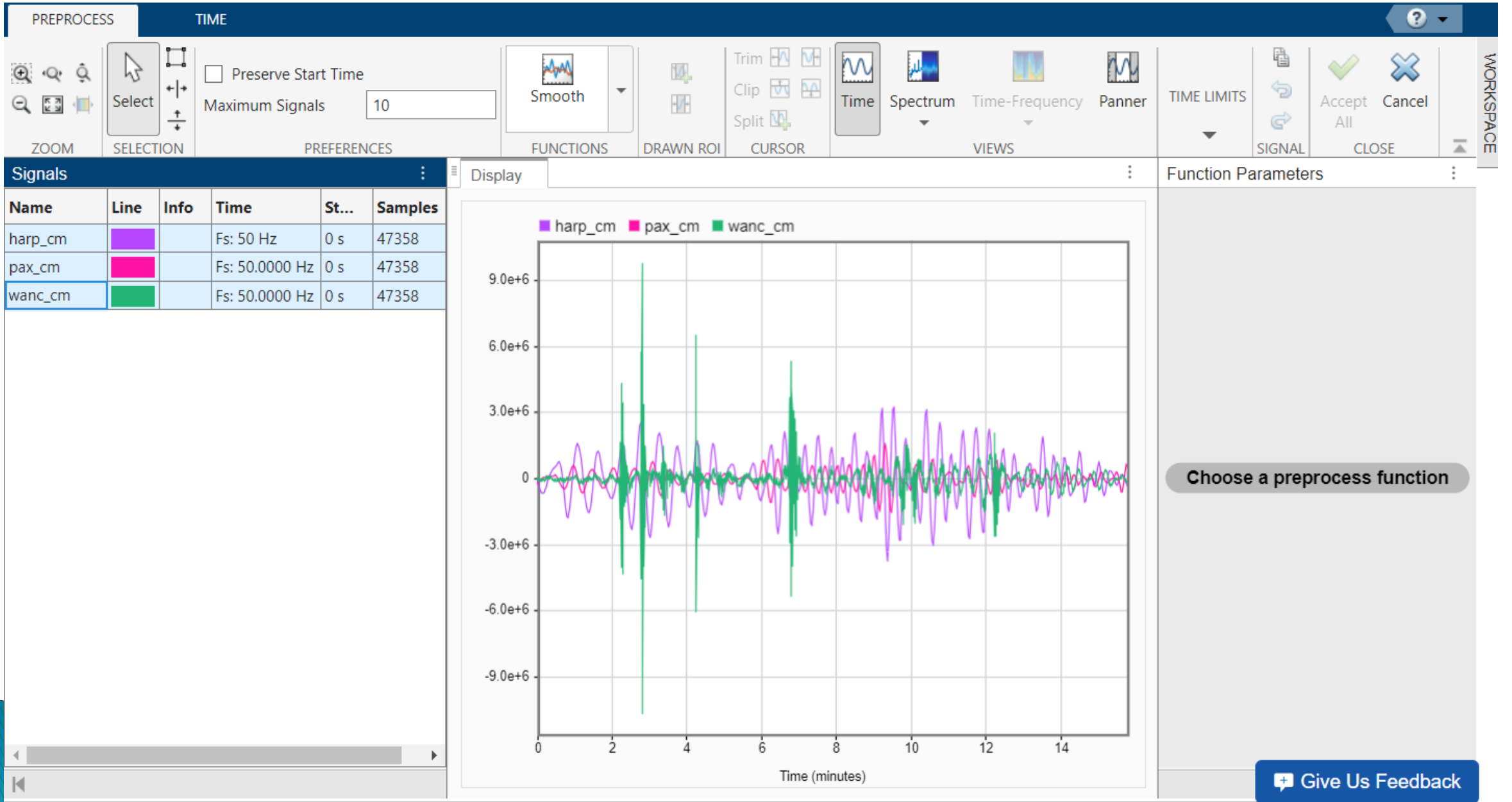
▼ CUSTOM FUNCTION

Function nm2cm
Arguments none

Help Apply

Give Us Feedback

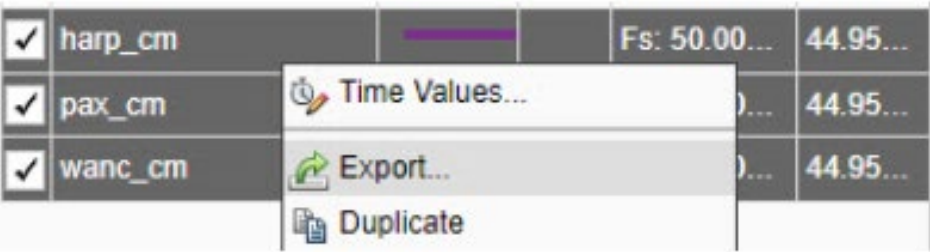
Task 1



Task 2

Instead of adding functionality to Signal Analyzer, you may want to process your signals **outside the app**. In the next activity, you'll align these three signals to make it easier to compare the peaks. This requires using MATLAB functions outside of the app because the functions require multiple signals. To continue your work outside of Signal Analyzer, you can export the signals.

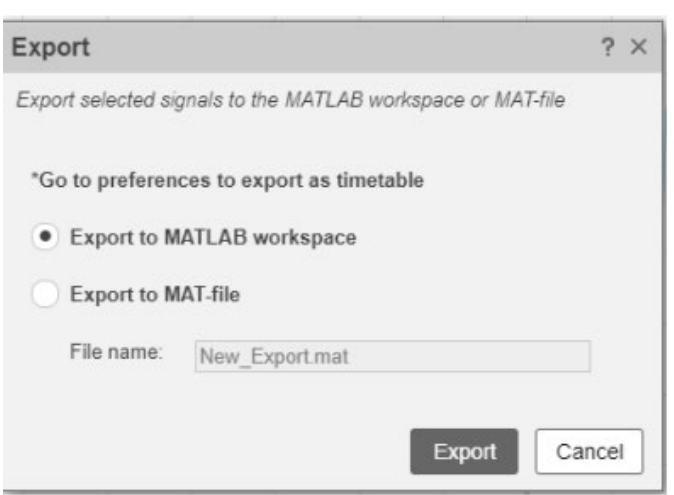
To export, right-click the signal and select Export.



#TASK

Export all three signals with centimeter units to the Workspace.

Name	Size	Class
 harp	240000×1	double
 harp_cm	47358×1	double
 pax	240000×1	double
 pax_cm	47358×1	double
 paxT	240000×1	double
 wanc	480000×1	double
 wanc_cm	47358×1	double



Align Signals

Task 1

Since all three seismic stations should have recorded the same surface waves, all three signals should be correlated. The `alignsignals` function uses cross-correlation to estimate and delay signals.

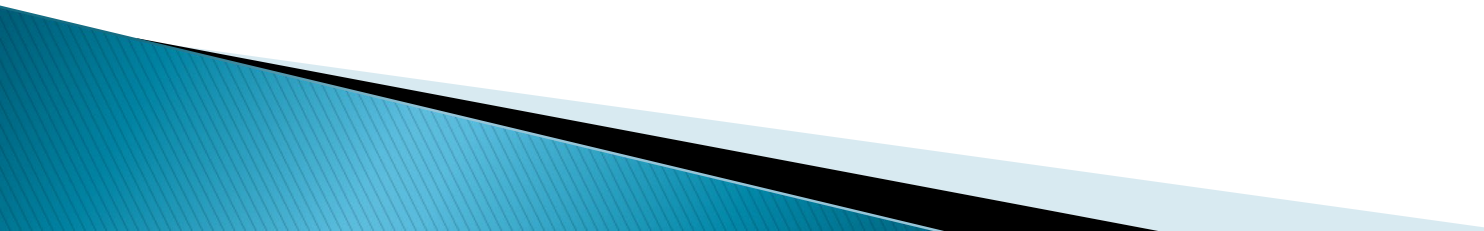
`[xa,ya] = alignsignals(x,y)`

If two signals have a **sufficient correlation**, the earlier output signal is **padded with zeros**.

#TASK

The signals with centimeter units were renamed back to **harp**, **pax**, and **wanc** to keep the variable names short.

Align harp and wanc. Use the same output variable names so that the visualization code at the bottom of the script plots the aligned signals.



Task 1

Align Signals

This code imports the signals.

load `processedsignals.mat`

Use the **finddelay function** to find the signal that arrived last.

`finddelay(harp,pax)`

`finddelay(harp,wanc)`

`finddelay(pax,wanc)`

Based on the delay values to the right, you can conclude that:

- harp arrived 596 samples after pax
 - harp arrived 1167 samples before wanc
 - pax arrived 1770 samples before wanc
- So the wanc signal arrived last.

Task 1

```
[harp,wanc] = alignsignals(harp,wanc);
```

```
ans = -596
```

```
ans = 1167
```

```
ans = 1770
```

Task 2

Align the **pax** and **wanc** signals. Use the same output variable names so that the visualization code at the bottom of the script plots the aligned signals.

Task 2

```
[pax,wanc] = alignsignals(pax,wanc);
```

%This code plots the signals from the HARP and PAX stations.

```
plot(harp)
```

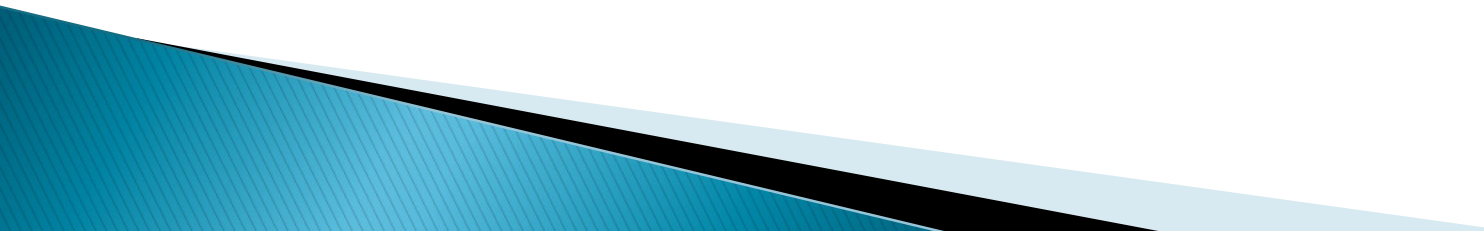
```
hold on
```

```
plot(pax)
```

```
plot(wanc)
```

```
hold off
```

```
legend("HARP","PAX","WANC")
```



In this plot, you can see the signals now **look aligned**.

Figure 4: Figure

File Edit View Insert Tools

